

Fourth development changes understanding code block by block

After completing the main functionality for the Manage Readings and View Readings pages, I focused on improving the overall structure, navigation and presentation of the frontend so that it felt like one complete application rather than a collection of separate screens. I created a reusable **shared Header component** that appears across the Home, Manage Readings and View Readings pages. The header contains the Cold Chain Operations Portal branding and navigation links, and it uses the current URL to visually identify which page the user is viewing. I added the Header to the root Next.js layout so that it only needed to be written once and would automatically appear throughout the application.

I then redesigned the **Home page** into a proper landing page for the Cold Chain Operations Portal. The page explains what the portal is used for, displays the permitted 2°C–8°C cold-chain temperature range, explains what an excursion means, provides a simple four-step explanation of how the system works, and gives the user clear links to the Manage Readings and View Readings sections. I followed the professional navy, white and cyan visual style I had chosen for the frontend so that the Home page was consistent with the rest of the application.

As part of the final improvements, I simplified some of the technical wording on the Home page. I removed unnecessary references to the "AWS backend" and instead referred simply to the **backend**, because the landing page is intended for users rather than as detailed technical documentation. I also replaced the original backend-information section at the bottom of the Home page with an **About the Developer** section. This introduces me as **Umar Asif Hussain, Junior Developer at Encon Pharma**, and explains that the Cold Chain Operations Portal is one of the projects I completed during my internship. The footer was also updated to include my name and to state that the application was **Developed with: Next.js • TypeScript • Tailwind • TanStack Query**.

I also improved the reusable notification system. The success, excursion and error notifications already displayed an `X` close button, but originally the button did not change any state, so clicking it had no effect. I updated the `AlertNotification` component to accept an optional `onClose` function and connected the button's `onClick` event to that function. I then updated the Manage Readings page so that each notification clears the correct state when closed. Validation messages clear their validation state, TanStack Query mutation messages use the mutation's `reset()` function, and the successful DELETE notification clears its stored deleted-reading UUID. This made the notifications properly interactive and improved the overall usability of the portal.

1. Creating the shared Header.tsx

I created:

```
1 | app/components/Header.tsx
```

rather than manually adding the same navigation bar to every page.

The main reason for doing this was **reusability**.

Without a shared component, the project could have ended up like:

```
1 | Home page
2 |   → own navigation code
3 |
4 | Manage page
5 |   → copied navigation code
6 |
7 | Readings page
8 |   → copied navigation code
```

That would create duplicated code.

Instead, the application now works like:

```
1 | layout.tsx
2 |   ↓
3 | Header.tsx
4 |   ↓
5 | Page content
```

The same Header therefore appears automatically on:

```
1 | /
2 | /manage
3 | /readings
```

This also means that if I decide to change the navigation later, I only need to update one component.

2. Why Header.tsx is a Client Component

The Header begins with:

```
1 | "use client";
```

This is required because I use:

```
1 | usePathname()
```

from Next.js.

`usePathname()` reads the current browser route so the component can determine which page is active.

For example:

```
1 /  
2 → Home active  
3  
4 /manage  
5 → Manage Readings active  
6  
7 /readings  
8 → View Readings active
```

Because the Header responds to the current browser location, it requires client-side functionality.

3. Navigation array

Instead of writing three separate Link components manually, I created a navigation array containing:

```
1 const navigation = [  
2   {  
3     name: "Home",  
4     href: "/",  
5   },  
6   {  
7     name: "Manage Readings",  
8     href: "/manage",  
9   },  
10  {  
11    name: "View Readings",  
12    href: "/readings",  
13  },  
14  ];
```

This keeps all of the navigation options together.

The component then uses:

```
1 navigation.map(...)
```

to create the links.

Conceptually:

```
1 navigation array  
2   ↓  
3 map()  
4   ↓  
5 Home  
6 Manage Readings  
7 View Readings
```

This makes it easier to add another page in the future because another object could simply be added to the array.

4. `usePathname()`

Inside the Header I use:

```
1 | const pathname = usePathname();
```

This stores the route currently being viewed.

Then, for each navigation link, I check:

```
1 | const isActive =  
2 |   pathname === item.href;
```

For example, when the user is at:

```
1 | /manage
```

the comparison becomes:

```
1 | /manage === /manage
```

so:

```
1 | isActive = true
```

The Manage Readings navigation link can then receive different styling.

5. Active navigation styling

If a page is currently active, the navigation link receives a cyan underline and stronger text styling.

If it is not active, it receives the normal styling but gains the cyan effect when hovered over.

This gives the user a clear visual indication of:

Where am I in the application?

I also used:

```
1 | aria-current="page"
```

for the active navigation item.

This is useful for accessibility because assistive technologies can understand that this link represents the current page.

6. Portal branding

The Header also contains a small:

```
1 | CC
```

logo-style block along with:

```
1 | Cold Chain  
2 | Operations Portal
```

This gives the frontend consistent application branding.

The entire brand section is also a Next.js `Link` back to:

```
1 | /
```

so the user can click the portal name to return to the Home page.

7. Using the Header in `layout.tsx`

Instead of importing the Header separately into every page, I added it to:

```
1 | app/layout.tsx
```

The structure is now approximately:

```
1 | <Providers>  
2 |   <Header />  
3 |   {children}  
4 | </Providers>
```

`children` represents whichever page is currently being rendered.

So Next.js effectively produces:

```
1 | Header  
2 | +  
3 | Home page
```

or:

```
1 | Header  
2 | +  
3 | Manage page
```

or:

```
1 | Header  
2 | +  
3 | Readings page
```

depending on the route.

This is one of the advantages of the Next.js App Router and shared layouts.

8. Updating the application metadata

I also changed the original starter metadata from:

```
1 | Create Next App
```

to:

```
1 | Cold Chain Operations Portal
```

with a description explaining that the application is for cold-chain temperature monitoring and management.

This removes the default Next.js starter branding and makes the browser metadata match the actual project.

9. Redesigning the Home page

I completely redesigned:

```
1 | app/page.tsx
```

from the original basic landing page.

The previous version only contained the portal heading and two buttons.

The new page is designed to explain:

```
1 | What the portal is
2 | What temperature range is permitted
3 | What an excursion is
4 | How the system works
5 | Where the user can manage readings
6 | Where the user can view readings
7 | Who developed the project
```

This makes the Home page useful even for someone who has never used the system before.

10. Hero section

At the top of the Home page I created a large navy hero section containing:

```
1 | Cold Chain Monitoring
2 |
3 | Cold Chain
4 | Operations Portal
```

and a short explanation:

```
1 | A central portal for recording,
2 | monitoring and managing cold-chain
3 | temperature readings across branches.
```

I also included two main actions:

- 1 Manage Readings
- 2 View Readings

These use Next.js `Link` components and allow the user to immediately navigate to the two functional parts of the frontend.

11. Professional visual design

I used a consistent design throughout the page based on the professional style I had chosen for the portal.

The main colours include:

- 1 Dark navy
- 2 White
- 3 Light grey
- 4 Cyan
- 5 Red for warnings

The page also uses:

- 1 Large headings
- 2 Rounded cards
- 3 Spacing between sections
- 4 Hover effects
- 5 Responsive grid layouts
- 6 Clear call-to-action buttons

This creates a much more polished interface than the original starter application.

12. About the Portal section

The next section explains that the portal allows users to:

- 1 Create
- 2 Update
- 3 Delete
- 4 Retrieve

temperature readings.

It also explains that each reading receives a unique UUID and is automatically checked against the cold-chain temperature rules.

This gives the user enough understanding of what happens when they submit a reading without showing unnecessary implementation detail.

13. Simplifying references to the backend

Originally some Home page text referred specifically to:

```
1 | AWS backend
```

I decided this was more technical than necessary for a user-facing Home page.

I changed the wording to simply:

```
1 | backend
```

For example:

```
1 | This operations portal connects  
2 | directly to the cold-chain backend...
```

This keeps the Home page focused on what the application does rather than the specific cloud platform underneath it.

The technical AWS details can still be documented separately in the project README or technical write-up.

14. Temperature rule card

I created a prominent temperature-rule card displaying:

```
1 | 2°C to 8°C
```

This represents the permitted temperature range used throughout the project.

The page explains that the reading remains safe when the recorded temperatures stay within that range.

It also contains a separate warning section:

```
1 | △ Excursion
```

which explains that:

```
1 | minimum < 2°C  
2 | OR  
3 | maximum > 8°C
```

causes a reading to become an excursion.

This gives users an immediate explanation of one of the most important business rules in the application.

15. How to Use the Portal section

I added a four-stage explanation of how the application works.

The stages are:

1. **Log a reading** — The user enters the branch, date/time, minimum temperature, maximum temperature and recorded-by information.
2. **Automatic validation** — The backend validates the request, generates the UUID and determines the reading status.
3. **Handle excursions** — Out-of-range readings are marked as excursions and trigger the alert process.
4. **Review the data** — Users can view all readings for a branch or retrieve one individual reading by UUID.

This gives a new user a simple walkthrough without requiring them to understand the internal code.

16. Main navigation cards

Further down the Home page I created two large navigation cards.

The first links to:

```
1 | /manage
```

and explains that the user can:

```
1 | Create
2 | Edit
3 | Delete
```

readings.

The second links to:

```
1 | /readings
```

and explains that the user can:

```
1 | View branch readings
2 | Retrieve one reading by UUID
```

This gives the user another clear navigation option after reading about how the portal works.

17. Replacing the backend-information card

Originally, the bottom section of the Home page contained a technical explanation of the backend architecture.

It included information similar to:

```
1 | Next.js
2 | ↓
```

```
3 TanStack Query
4 ↓
5 Next.js API Proxy
6 ↓
7 API Gateway
8 ↓
9 Lambda
10 ↓
11 DynamoDB
```

I decided this information was unnecessary on the main landing page.

The Home page is intended to introduce the application, not act as a technical architecture document.

I therefore replaced this section with an:

```
1 | About the Developer
```

section.

18. About the Developer section

The new section displays:

```
1 | Umar Asif Hussain
2 | Junior Developer at Encon Pharma
```

and explains that the Cold Chain Operations Portal is one of the projects I completed during my internship.

The section also describes how the project allowed me to combine the different skills I developed during the internship, including:

```
1 | software development
2 | cloud
3 | infrastructure
4 | API development
5 | frontend development
```

into one complete application.

This makes the Home page feel more personal and also gives context to why the project was created.

19. Explaining the full-stack project

The About section also explains that I developed the frontend to work with the backend application I had already built.

This demonstrates that the final result is not simply a standalone user interface.

Instead:

```
1 Backend project
2   +
3 Frontend project
4   ↓
5 Complete application
```

The frontend performs real operations against the backend rather than using demonstration data.

20. Updating the footer

I also redesigned the footer.

It now displays:

```
1 Umar Asif Hussain
2
3 Cold Chain Operations Portal
```

which clearly identifies both the developer and the project.

On the other side it displays:

```
1 Developed with:
2 Next.js • TypeScript • Tailwind • TanStack Query
```

This replaced the previous generic technology text.

Using:

```
1 Developed with:
```

makes it clear that these are the technologies used to build the frontend.

21. Improving `AlertNotification.tsx`

Another usability issue I corrected involved the notification pop-ups.

The application uses:

```
1 app/components/AlertNotification.tsx
```

for:

```
1 Success
2 Error
3 Excursion
```

notifications.

Although the notification visually contained an `X` button, initially clicking it did not remove the notification.

The reason was simple:

```
1 | Button existed visually
2 | but
3 | button had no state-changing action
```

22. Adding `onClose`

I updated the component props to include:

```
1 | onClose?: () => void;
```

This means the parent component can optionally provide a function that should run when the user closes the notification.

The `?` means:

```
1 | optional
```

so the component would still work even if no close function were provided.

23. Connecting the close button

The actual button now uses:

```
1 | onClick={onClose}
```

So the flow becomes:

```
1 | User clicks ×
2 |   ↓
3 | onClose runs
4 |   ↓
5 | Parent state changes
6 |   ↓
7 | React re-renders
8 |   ↓
9 | Notification disappears
```

The close button also has:

```
1 | aria-label="Close notification"
```

so its purpose can be understood by accessibility tools.

24. Closing validation errors

Validation messages are based on React state.

For example:

```
1 | createValidationError
```

So the Create validation notification uses:

```
1 | onClose={() =>
2 |   setCreateValidationError("")
3 | }
```

This changes the error string back to an empty string.

The interface only displays the notification when:

```
1 | createValidationError
```

contains a value.

Therefore:

```
1 | Error exists
2 | → notification displayed
3 |
4 | User clicks ×
5 | → error becomes ""
6 |
7 | No error value
8 | → notification removed
```

25. Resetting TanStack mutations

Success and API error notifications are based on the status of the TanStack Query mutation.

For example:

```
1 | createMutation.isSuccess
```

or:

```
1 | createMutation.isError
```

For these notifications I use:

```
1 | createMutation.reset()
```

when the close button is clicked.

The same approach is used for:

```
1 | updateMutation.reset()
2 | deleteMutation.reset()
```

where appropriate.

`reset()` clears the mutation's result and error state.

So:

```
1 Mutation succeeds
2   ↓
3 isSuccess = true
4   ↓
5 success notification shown
6   ↓
7 user presses ×
8   ↓
9 mutation.reset()
10  ↓
11 isSuccess reset
12  ↓
13 notification disappears
```

26. Closing the DELETE success notification

The successful DELETE notification works slightly differently because I store the UUID of the deleted reading in:

```
1 deletedReadingId
```

The notification therefore closes using:

```
1 setDeletedReadingId("")
```

Once the UUID is cleared, React no longer renders the success message.

27. Why the notification improvement matters

This was a relatively small code change but an important usability improvement.

Before:

```
1 Notification appears
2   ↓
3 User clicks ×
4   ↓
5 Nothing happens
```

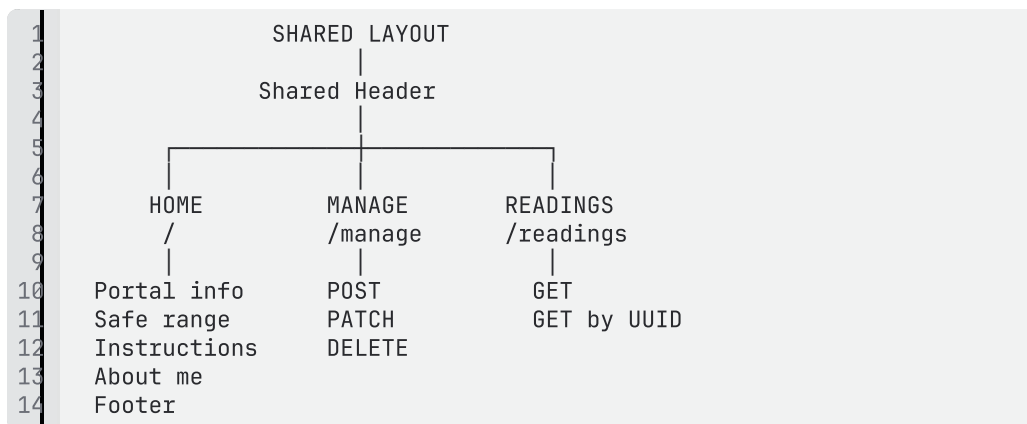
After:

```
1 Notification appears
2   ↓
3 User reads message
4   ↓
5 User clicks ×
6   ↓
7 Notification closes
```

This makes the application behave in the way a user would naturally expect.

Final frontend structure after these improvements

The application now has this overall structure:



The Header provides navigation between all three areas, while the Home page acts as the central introduction to the system.

The Manage and Readings pages remain focused on the actual operational functionality.

Recent frontend improvements completed

These latest changes completed the presentation and usability side of the portal:

- 1 ☒ Shared Header component
- 2 ☒ Header appears on every page
- 3 ☒ Home navigation
- 4 ☒ Manage navigation
- 5 ☒ View Readings navigation
- 6
- 7 ☒ Active page highlighting
- 8 ☒ usePathname used
- 9 ☒ aria-current added
- 10 ☒ Shared Next.js layout used
- 11
- 12 ☒ Default Next.js metadata replaced
- 13 ☒ Portal title and description added
- 14
- 15 ☒ Complete Home page redesign
- 16 ☒ Professional navy/white/cyan style
- 17 ☒ Hero section
- 18 ☒ Portal explanation
- 19 ☒ Safe 2°C-8°C range displayed
- 20 ☒ Excursion explanation
- 21 ☒ Four-step usage guide
- 22 ☒ Manage navigation card
- 23 ☒ Readings navigation card
- 24
- 25 ☒ Unnecessary "AWS backend" wording removed
- 26 ☒ Simplified to "backend"
- 27
- 28 ☒ Technical backend card removed
- 29 ☒ About the Developer section added
- 30 ☒ Umar Asif Hussain added
- 31 ☒ Junior Developer at Encon Pharma added
- 32 ☒ Internship project explanation added
- 33
- 34 ☒ Footer updated
- 35 ☒ Developer name included

```
36 ✓ "Developed with:" wording added
37 ✓ Next.js
38 ✓ TypeScript
39 ✓ Tailwind
40 ✓ TanStack Query
41
42 ✓ Notification close buttons fixed
43 ✓ onClose prop added
44 ✓ Validation notifications dismiss correctly
45 ✓ Success notifications dismiss correctly
46 ✓ Error notifications dismiss correctly
47 ✓ Excursion notifications dismiss correctly
48 ✓ DELETE success notification dismisses correctly
```

These changes did not alter the underlying backend functionality. They focused on making the frontend feel like a **finished, consistent and professional Cold Chain Operations Portal** rather than simply a set of functional development screens.